

1 OOP

1.1 Type signature mechanics

Type signature: ordered type sequence of parameters.

- **NOT** part of the signature: return type, parameter names.
- **IS** part of the signature: parameter types and their order.

Trap: `int add(int X, int Y)` and `void add(int A, int B)` have the *same* signature (`int, int`) – return-type/name differences alone don't make them overloads. `m(int, double) vs m(double, int)` *do* differ.

1.2 Override vs. overload

- **Override:** same name, **same signature**, return type same or co-variant; subclass re-implements parent method.
- **Overload:** same name, **different signatures**; multiple methods coexist (same class or across class hierarchy).

Trap: `read():String` vs `read(int):String` = *overload*, not *override* (different signatures).

Binding

Override resolved by **dynamic binding** (actual object's runtime type). **Overload** resolved by **static binding** (declared compile-time types of arguments).

1.3 Polymorphism: 3-concept combo

Polymorphism only fires when *all three* hold:

1. **Substitutability:** caller code uses the parent type.
2. **Override:** subclass re-implements the operation.
3. **Dynamic binding:** language resolves the call at runtime.

No polymorphism if: the operation is overloaded (static binding), or the parent reference holds the parent type and no override exists, or the language resolves statically.

Substitutability direction

`Staff s = new AcademicStaff();` **OK** (subtype → supertype).

`AcademicStaff a = s;` **compile error** when `s` is declared `Staff` – needs explicit downcast.

1.4 Class / Abstract class / Interface

- **Class:** behavior spec + full implementation.
- **Abstract class:** behavior spec + *possibly incomplete* implementation; cannot be instantiated.
- **Interface:** behavior spec, *no* implementation.

Has ≥ 1 abstract method ⇒ class must be abstract.

Multiplicity in code

- `O..1` – plain instance variable (may be `null`).
- `1` – instance variable, must be initialised (compulsory association).
- `*` or `O..*` – collection (Array, List, Set, Map).

Composition vs. aggregation

Composition (filled diamond) implies: whole destroyed ⇒ parts destroyed; *no* cyclical whole-part links. **Cascading delete alone is not sufficient** – also need contextual whole-part identity. **Aggregation** (open diamond) is weaker: contained survives container.

2 Requirements

2.1 Use case extension numbering

(Worked example on ref sheet; the *rules* are not.)

- **3a., 3b.:** alternative flow that branches *after* MSS step 3.
- ***a.:** can occur at *any* step (e.g. user cancels).
- **Sub-steps** (3a1., 3a2.): expand the extension.
- **Resume:** extension ends with “Use case resumes from step *N*” or “Use case ends.”

2 Step content rules

A step gives the actor's **intention**, not the UI mechanics. **Bad:** “User right-clicks the text box and chooses *clear*.” **Good:** “User clears the input.”

MSS / extensions / preconditions / guarantees

- **MSS:** nothing-goes-wrong path; must be self-contained.
- **Extensions:** alternative/exceptional flows added on top.
- **Preconditions:** required system state *before* the use case starts.
- **Guarantees:** what the use case promises *after*.

Actor flexibility

One actor → many use cases; one use case → many actors. One person can play many roles; many people can fill one role. Actors are *outside* the system.

2.2 User stories vs. use cases

User stories are lighter, planning-oriented; **use cases** are detailed enough to drive interaction design. **{benefit} is droppable** when obvious.

Hierarchy

Epic / theme = high-level user story that decomposes into smaller ones.

Priority tiers

Essential (fail without) / **Typical** (peer products have) / **Novel** (differentiator).

3 Design

3.1 Coupling sub-types

(Generic def. on ref sheet; sub-types are not.)

- **Content:** one module modifies/reads another's internals directly.
- **Common/Global:** modules share global data.
- **Control:** one module passes a flag controlling another's flow.
- **External:** modules depend on externally-imposed convention (format, protocol).
- **Subclass:** child coupled to parent via inheritance.
- **Temporal:** actions grouped only because they happen at the same time.
- **Data:** modules share data through parameters only.

3.2 Cohesion forms

(Generic def. on ref sheet; higher = better.)

- **Single concept:** all code addresses one concept (e.g. `Student` handles only student data).
- **Same data:** code that manipulates the same data structure stays together.
- **Temporal:** code invoked close together in time (e.g. init routines).
- **NOTE:** mixing DB writes + UI dialogs in one class = low cohesion violation.

3.3 SOLID: smell-triggers

(What tells you the principle is broken)

- **SRP:** class changes for >1 distinct reason (e.g. `TextUi` parses and renders).
- **OCP:** adding a new type forces edits inside existing classes (not just new subclasses).
- **LSP:** subclass method is more restrictive than superclass spec (narrows valid inputs or widens exceptions).
- **ISP:** a client is forced to depend on an interface it only partially uses (fat interface – split it).
- **DIP:** high-level module `imports` a concrete low-level class (should depend on an abstraction/interface).

2 Law of Demeter (LoD)

“Don't talk to strangers.” Method `m` of object `O` may call methods only on:

- `O` itself; parameters of `m`; objects created inside `m`; direct attributes of `O`.

Violation: `b.getGoo().doSomething() – getGoo()` returns a stranger.

3.4 Other principles

- **YAGNI:** do not add code for potential future needs.
- **DRY:** every piece of knowledge has one authoritative representation (two different implementations of the same logic = DRY violation).
- **Brooks' law:** adding people to a late project makes it later (communication overhead > added output).

3.5 Design pattern intent

(Official sheet has structures; this is why you reach for each)

- **Singleton:** guarantee at most one instance; avoid accidental duplicates with real consequences.
- **Facade:** expose a component's services without leaking its internals.
- **Command:** represent a request as an object so it can be queued/stored/executed without knowing its type.
- **MVC:** decouple data (Model), display (View), and input handling (Controller) to reduce coupling from their interlinked nature.
- **Observer:** let ≥1 objects react to changes in another without the observed object depending on the observers.
- **Abstraction Occurrence:** split common info («Abstraction») from unique info («Occurrence») to avoid data duplication across similar entities.

Singleton cons: acts as global variable (raises coupling); hard to stub in tests (static method, not overridable).

3.6 Design approaches

- **Top-down:** high-level first; use for large/novel systems where architecture must stabilise early.
- **Bottom-up:** components first; use only for variations of existing systems or when reusing components.
- **Agile/emergent:** minimal upfront architecture; design evolves with requirements.

4 Modelling

4.1 Model purpose

Model: simpler view of a complex entity; captures only a selected aspect (omission = abstraction). Multiple models of the same entity may be needed to capture it fully.

- **Analyze:** understand problem domain or plan solution (e.g. architecture diagram).
- **Communicate:** share design among stakeholders (class diagram reverse-engineered from code; use case diagram to customer).
- **Blueprint:** guide construction of software.

4.2 CCD vs. Class Diagram

Conceptual Class Diagram (OODM): models the problem domain, not the solution.

- **Omits:** methods and navigability (both legal in a class diagram).
- **Excludes:** solution-specific classes (e.g. `DatabaseConnection`, `Logger`).
- **Captures:** class *structure* of the domain, not behavior.

MCQ trap: a CCD with methods or navigability arrows is notation-incorrect. A class diagram of domain classes is not automatically a CCD.

4.3 Sequence diagram reading traps

Frame semantics (official sheet shows syntax only):

- **alt:** exactly one branch executes – the first whose guard is true; `[else]` runs if no other guard matches.
- **opt:** the enclosed fragment executes only if the guard is true; nothing executes otherwise (= `alt` with one branch).
- **loop:** repeats while guard holds; guard checked before each iteration.
- **par:** all fragments execute concurrently (interleaved); order within a fragment is preserved.
- **ref:** replaced by the named interaction; all messages inside the referenced diagram execute at that point.

Self-call activation-bar nesting

A self-call creates a *new, nested* activation bar on the same lifeline – the outer bar must still be visible behind it. **MCQ trap:** drawing only one bar for a recursive/self-call = wrong; each call level = one bar.

Object-diagram underline rule

Instance names must be underlined in object diagrams (`obj : ClassName`). Without the underline the box is ambiguous with a class box. Attribute compartment shows `attribute = value`, not type declarations.

5 Refactoring

5.1 What it is *not*

- **Not rewriting:** discard-and-rewrite is not refactoring (must be small steps).
- **Not bug-fixing:** changes external behavior ⇒ no longer a refactoring.
- **Not free:** each small step needs regression testing – IDE-assisted refactors can still regress.

5.2 Catalogue: trigger → refactoring

(Names on ref sheet; trigger column is what's missing.)

- Same fragment in all branches → **Consolidate Conditional Expression**.
- One `if` doing two distinct things → **Decompose Conditional**.
- Method body is just as clear as the call → **Inline Method**.
- `!isEmpty()` or similar → **Remove Double Negative**.
- Numeric/string literal with implicit meaning → **Replace Magic Literal** (with named constant).
- Deep nesting where early-exit suffices → **Replace Nested Conditional with Guard Clauses**.
- Boolean/enum parameter dispatching different behaviors → **Replace Parameter with Explicit Methods**.
- `if (~cond)` where `if (cond)` is clearer → **Reverse Conditional**.
- One loop computing two unrelated things → **Split Loop**.
- One temp variable reused for two purposes → **Split Temporary Variable**.
- Code fragment groupable + nameable → **Extract Method**.

NOTE: some refactorings are opposites (`Extract` ↔ `Inline`).

5.3 Code smells (refactoring triggers)

- **Long Method, Large Class:** do too much.
- **Primitive Obsession:** ints/strings where a small type would carry meaning.
- **Temporary Field:** instance var only used in some flows.
- **Shotgun Surgery:** one logical change forces edits in many classes.
- **Data class:** only fields + getters, no behavior.

When to stop
Refactor while it pays down **technical debt**; stop when benefits stop exceeding cost.

6 Error handling

6.1 Exception vs. assertion

(*Defs on ref sheet; the decision is not.*)

- Exception**: unusual condition from user/environment (file missing, network drop) – caller can plausibly recover.
- Assertion failure**: programmer mistake – should terminate; assertions verify that the code itself is correct.

Use **both, complementarity**. Use `return` (not exceptions) for *normal* conditions; reserve exceptions for the unusual.

Assertion gotchas

- Disabled by default** (Java) – must enable at runtime; production code that requires the side effect will silently break.
- Never put work inside**: `assert writeFile()` skips the call when assertions are off.
- Suitable for: invariants (internal/control-flow/class), pre- and postconditions.

6.2 Defensive vs. Design-by-Contract

- Defensive**: callee *checks* preconditions and throws/handles violations. Slower, more code, safer at boundaries.
- DbC**: caller *guarantees* preconditions; callee assumes them. Faster, brittle if caller misbehaves. Use assertions to verify (debug-mode safety net).

Tune defensiveness by: system criticality, who calls the code (you vs. others), language support, overhead budget.

Logging

Deliberate recording for later forensic use – does *not* prevent failures; helps diagnose them. Most systems support log levels (e.g. INFO, WARNING, SEVERE) so noise can be filtered.

7 Integration

7.1 Three orthogonal axes

(*Names on ref sheet; the why and the test-double mapping are not.*)

- Frequency**: *late + one-time* (wait till end) vs. *early + frequent* (✓ – surfaces incompatibilities one at a time).
- Granularity**: *big-bang* (all at once) vs. *incremental* (✓ – manageable failure isolation).
- Direction**: *top-down / bottom-up / sandwich*.

Direction → **test-double needed**

- Top-down**: needs **stubs** (lower-level not yet ready). Surfaces high-level problems early.
- Bottom-up**: needs **drivers** (UI/top-level not ready) to invoke the lower components.
- Sandwich**: do both, meet in the middle.

CI vs. CD

CI: build + test on every change. **CD**: *also* deploys to users automatically. **CI is a prerequisite for CD**, not a synonym.

8 Quality Assurance

8.1 EP derivation procedure

EPs derived from the SUT's specification. Steps:

- Identify all **data participants**: input parameters, target object state, any global data accessed.
- For each participant, group inputs that the SUT processes *the same way* into one partition.

- If a parameter has only a few discrete values with distinct behavior, make **each value its own partition** (e.g. enums, flags "F"/"T"/"D").
- If adjacency is meaningless (e.g. prime vs. non-prime), note that EPs need not be contiguous.
- Pick **one representative** from each partition per test case.
Example: `isValidMonth(m)`: partitions `[MIN_INT..0]`, `[1..12]`, `[13..MAX_INT]`.

8.2 BVA: which boundary values?

BVA rationale: bugs cluster at boundaries because off-by-one errors are common (`<` vs. `<=`, `>` vs. `>=`). For each EP boundary, test **just below, on, and just above**; prioritise these over interior values.

- Numeric range** `[1, 12]`: test 0, 1, 2 ... 11, **12**, 13.
- String length**: empty string, string of max allowable length.
- Collection size**: empty, one element, one-from-full, full.

8.3 Combining inputs: which strategy?

- All combinations**: use only if $N_{partitions}$ is tiny; guarantees full coverage but explodes fast.
- All pairs**: default for multi-input methods; catches most interaction bugs with far fewer cases than all-combinations.
- At least once**: minimum viable – use when time is extremely tight or inputs are truly independent.
- Random subset**: use on top of another strategy to probe untested combinations cheaply.

Heuristic rationales

- Each valid input at least once in a positive test**: if a valid input only appears in negative (error) test cases, its correct behavior is never verified.
- Test invalid inputs individually**: combining two invalid inputs masks which one triggered the error – you cannot tell which is responsible.

Typical workflow

Apply *at-least-once* first → enforce valid-input heuristic → isolate invalid inputs → add hidden-interaction cases.

Worked: `duplicate(String s, int n)`
EPs: `s`: {zero-length}, {contains whitespace}, {normal} and `n`: {negative}, {0}, {positive}. BVA adds: `n = 0` (boundary between negative/zero), `n = 1` (zero/positive), `n = MAX_INT`. One negative test: `n = -1` alone (do not pair with invalid `s` simultaneously).

9 Process models

9.1 When to pick which?

(*Names on ref sheet; the decision criteria are not.*)

- Sequential (waterfall)**: requirements stable + well-understood; effort estimable; progress tracked by stage.
- Iterative – breadth-first**: working product needed each iteration; risk: shallow first cuts.
- Iterative – depth-first**: high-risk components needed first; risk: no working product early.
- Agile (XP, Scrum)**: requirements volatile; close customer collaboration; tolerate evolving design.
- Unified Process**: large project needing customisable iteration count + phase intensities.

Mix is normal: depth-first early iterations → breadth-first later; sequential within iterations.

9.2 Scrum mechanics (MCQ trap)

- Roles**: Scrum Master (process), Product Owner (stakeholders), Team (cross-functional).
- Sprint**: fixed length (1 wk–1 mo). Backlog **freezes** once sprint starts.
- Timeboxed**: unfinished features return to backlog, never spill into next sprint.
- Daily scrum**: 15 min, 3 questions: did yesterday / will do today / impediments. Issues resolved *offline* after, not in the meeting.

Unified Process: 4 phases

Inception (vision, rough plan) → **Elaboration** (architecture, design) → **Construction** (implementation, testing) → **Transition** (deploy, train users).

Agile manifesto: 4 values

Individuals + interactions > processes/tools; working software > docs; customer collab > contract; respond to change > follow plan.

10 Git mechanics

10.1 Fast-forward: when?

FF merge happens iff the destination has *not diverged* from the source – i.e. destination's HEAD is an ancestor of source's HEAD.

- Diverged** ⇒ **no FF**: a real merge commit is created (or merge is rejected with `-ff-only`).
- No FF wanted? Use `-no-ff`**: forces a merge commit to preserve branch shape in history.

10.2 pull vs. pull -rebase

- git pull** = `fetch + merge`: creates a merge commit when local and remote have diverged.
- git pull -rebase** = `fetch + rebase`: replays your local commits on top of the fetched remote → *linear* history, no merge commit.

Rebase rewrites history: commit SHAs change. **Never rebase commits already pushed to a shared branch.**

10.3 Merge / rebase / squash compared

- Merge**: both histories preserved; merge commit ties them.
- Rebase**: source commits replayed on dest; SHAs change; linear history.
- Squash merge**: *one* new commit on dest combining all source commits; individual source commits not on dest. Source branch unchanged.
- Cherry-pick**: copy a single commit from one branch to another (new SHA).

Stash

stash apply: copy stash to working tree, *keep* stash entry.
stash pop: apply, then *drop* the entry.

Detached HEAD

HEAD points directly to a commit (not a branch). New commits are **not reachable** from any branch – checking out anything else strands them. Use `git switch -c <name>` to anchor.

Tag types

Lightweight: a name pointing to a commit (no metadata). **Annotated**: full Git object – has tagger, date, message, can be GPG-signed; required for releases.